

National Health and Nutrition Examination Survey (NHANES)

Luck B. Simelus

2026-09-19

1 - Introduction

The National Health and Nutrition Examination Survey (NHANES) is a health survey conducted by the National Center for Health Statistics (NCHS), part of the Centers for Disease Control and Prevention (CDC). NHANES collects information about the health and nutritional status of people in the United States through interviews, physical examinations, and laboratory tests. The data collected through NHANES can be used to study different health conditions and risk factors in the U.S. population.

For this exercise, I worked with NHANES data from three survey cycles: 2013–2014, 2015–2016, and 2017–2018. The focus was on demographic information and blood pressure measurements. The main goal was to practice using R to combine, clean, and organize data from different datasets. The data were merged using the participant identification number, and blood pressure measurements with validity issues were handled before preparing the data for analysis. Finally, the dataset was reshaped from wide to long format. These steps provided an opportunity to practice basic data manipulation and tidying techniques in R using a real-world public health dataset.

Loading packages

We will first install all the packages needed for importing and manipulating the data

```
library(pacman)
pacman::p_load(rio, here, janitor, dplyr, readr, tidyverse, skimr)
```

2 - Importing the datasets

```
demographics_data_2013_2014 <- import(here("data", "DEMO_H.xpt"))
blood_pressure_data_2013_2014 <- import(here("data", "BPX_H.xpt"))

demographics_data_2015_2016 <- import(here("data", "DEMO_I.xpt"))
blood_pressure_data_2015_2016 <- import(here("data", "BPX_I.xpt"))

demographics_data_2017_2018 <- import(here("data", "DEMO_J.xpt"))
blood_pressure_data_2017_2018 <- import(here("data", "BPX_J.xpt"))
```

Initial review of the datasets

```
### Initial review of the datasets
skim(demographics_data_2013_2014)
skim(blood_pressure_data_2013_2014)

skim(demographics_data_2015_2016)
skim(blood_pressure_data_2015_2016)

skim(demographics_data_2017_2018)
skim(blood_pressure_data_2017_2018)
```

Table 1 : Datasets dimension

Dataset	Number of Row	Number of Columns
demographics_data_2013_2014	10175	47
blood_pressure_data_2013_2014	9813	23
demographics_data_2015_2016	9971	47
blood_pressure_data_2015_2016	9544	21
demographics_data_2017_2018	9254	46
blood_pressure_data_2017_2018	8704	21

The variables RIAGENDR and DMDEDUC2 are not in the right format. They are in the numerical format. A more appropriate format would be factor which would represent the categories better.

3 - Merging and preparing the datasets

In this section, we are first going to select the columns needed in each of the datasets and merge the resulting datasets into one full dataset that we can easily work with

Blood pressure

```
# Select the columns related to systolic and diastolic blood pressure
blood_pressure_sys_di_2013_2014 <- blood_pressure_data_2013_2014 %>%
  select(SEQN, PEASCCT1, BPXSY1, BPXDI1, BPXSY2,
         BPXDI2, BPXSY3, BPXDI3, BPXSY4, BPXDI4)

blood_pressure_sys_di_2015_2016 <- blood_pressure_data_2015_2016 %>%
  select(SEQN, PEASCCT1, BPXSY1, BPXDI1, BPXSY2,
         BPXDI2, BPXSY3, BPXDI3, BPXSY4, BPXDI4)

blood_pressure_sys_di_2017_2018 <- blood_pressure_data_2017_2018 %>%
  select(SEQN, PEASCCT1, BPXSY1, BPXDI1, BPXSY2,
         BPXDI2, BPXSY3, BPXDI3, BPXSY4, BPXDI4)
```

```
# Combine the 3 waves of blood_pressure_data into one dataset
blood_pressure_sys_di_all_waves <- bind_rows(
  blood_pressure_sys_di_2013_2014 %>% mutate(wave_id = "2013-2014"),
  blood_pressure_sys_di_2015_2016 %>% mutate(wave_id = "2015-2016"),
  blood_pressure_sys_di_2017_2018 %>% mutate(wave_id = "2017-2018")
)
```

The resulting dataset *blood_pressure_sys_di_all_waves* has 28061 rows and 11 columns

Demographic

```
# select the columns needed in the demographics datasets
demo_data_2013_2014 <- demographics_data_2013_2014 %>%
  select( SEQN, RIAGENDR, RIDAGEYR, RIDRETH1, RIDRETH3,
    DMDDEDUC2, DMDMARTL, DMDFMSIZ, INDFMIN2)

demo_data_2015_2016 <- demographics_data_2015_2016 %>%
  select( SEQN, RIAGENDR, RIDAGEYR, RIDRETH1, RIDRETH3,
    DMDDEDUC2, DMDMARTL, DMDFMSIZ, INDFMIN2)

demo_data_2017_2018 <- demographics_data_2017_2018 %>%
  select( SEQN, RIAGENDR, RIDAGEYR, RIDRETH1, RIDRETH3,
    DMDDEDUC2, DMDMARTL, DMDFMSIZ, INDFMIN2)

# Combine the 3 waves of demoraphics data into one dataset
demo_data_all_waves <- bind_rows(
  demo_data_2013_2014 %>% mutate(wave_id = "2013-2014"),
  demo_data_2015_2016 %>% mutate(wave_id = "2015-2016"),
  demo_data_2017_2018 %>% mutate(wave_id = "2017-2018")
)
```

The resulting dataset *demo_data_all_waves* has 29400 rows and 10 columns

Final merged dataset

```
# Merge the 2 combined datasets (demo and blood pressure ) into one full data
set
demo_bp_all <- full_join(
  demo_data_all_waves, blood_pressure_sys_di_all_waves,
  by = "SEQN"
)
# Dimension of the final dataset
dim(demo_bp_all)
```

The merged dataset contains 29400 rows, the same number as the combined demographic dataset because it preserves all observations from the demographic data. Even the ones that do not have any matching SEQN in the blood pressure dataset.

It contains 20 columns because the 10 demographic variables and 11 blood-pressure variables share the SEQN identifier, which is retained only once in the merged dataset ($10 + 11 - 1 = 20$).

4 - Data cleaning and variable recoding

Now that we have one merge dataset, we will clean the data and recode the variables as needed so that it will be ready for analysis.

Step 1 : Review the data

```
skim(demo_bp_all)
```

There are 2 character columns and 18 numeric columns. The two character columns are the wave_id column we created for each of the combined demographics and blood pressure datasets. We will need to keep only one of them, the one from the demographic datasets (wave_id.x) since it will be the one without any missing value

All the column coming from the blood pressure datasets have missing value for the patients that are in the demographic dataset but do not have any matching SEQN in the blood pressure dataset.

There are also variables from the demographic dataset with missing values: DMDEDUC2, DMDMARTIL, INDFMIN2.

Step 2 : Clean column names

In this step we are going to clean and rename the column and drop the column wave_id.y that is a duplicate of the wave_id.x column.

```
# automatic column names cleaning
demo_bp_all_cleaned <- demo_bp_all %>%
  clean_names()
# keeping one column for wave id and renaming columns
demo_bp_all_cleaned <- demo_bp_all_cleaned %>%
  select(-wave_id_y) %>%
  rename(wave_id = wave_id_x,
         sex = riagendr,
         age = ridageyr,
         ethnicity = ridreth1,
         race = ridreth3,
         educ_level = dmddeduc2,
         marital_status = dmdmart1,
         family_size = dmdfmsiz,
         family_income = indfmin2,
         bpx_comment = peascct1)
# Verify that all the columns are renamed properly
colnames(demo_bp_all_cleaned)
```

Step 3: Standardize and re-code variables

Let's transform all variables into the appropriate type and re-code them so that they have meaningful categories

```
demo_bp_all_recoded <- demo_bp_all_cleaned %>%  
  mutate(  
    sex = recode(  
      sex,  
      "1" = "Male",  
      "2" = "Female"  
    ),  
    ethnicity = recode(  
      ethnicity,  
      "1" = "Mexican american",  
      "2" = "Other hispanic",  
      "3" = "Non-Hispanic White",  
      "4" = "Non-Hispanic Black",  
      "5" = "Other race - Including Multi-Racial"  
    ),  
    race = recode(  
      race,  
      "1" = "Mexican american",  
      "2" = "Other hispanic",  
      "3" = "Non-Hispanic White",  
      "4" = "Non-Hispanic Black",  
      "6" = "Non-Hispanic Asian",  
      "7" = "Other race - Including Multi-Racial"  
    ),  
    educ_level = recode(  
      educ_level,  
      "1" = "Less than 9th grade",  
      "2" = "9-11th grade (Includes 12th grade with no diploma)",  
      "3" = "High school graduate/GED or equivalent",  
      "4" = "Some college or AA degree",  
      "5" = "College graduate or above",  
      "7" = "Refused",  
      "9" = "Don't Know"  
    ),  
    marital_status = recode(  
      marital_status,  
      "1" = "Married",  
      "2" = "Widowed",  
      "3" = "Divorced",  
      "4" = "Separated",  
      "5" = "Never married",  
      "6" = "Living with partner",  
      "77" = "Refused",
```

```

    "99" = "Don't Know"
  ),
  family_income = recode(
    family_income,
    "1" = "$ 0 to $ 4,999",
    "2" = "$ 5,000 to $ 9,999",
    "3" = "$10,000 to $14,999",
    "4" = "$15,000 to $19,999",
    "5" = "$20,000 to $24,999",
    "6" = "$25,000 to $34,999",
    "7" = "$35,000 to $44,999",
    "8" = "$45,000 to $54,999",
    "9" = "$55,000 to $64,999",
    "10" = "$65,000 to $74,999",
    "12" = "$20,000 and Over",
    "13" = "Under $20,000",
    "14" = "$75,000 to $99,999",
    "15" = "$100,000 and Over",
    "77" = "Refused",
    "99" = "Don't Know"
  ),
  family_size = recode(
    family_size,
    "1" = "1",
    "2" = "2",
    "3" = "3",
    "4" = "4",
    "5" = "5",
    "6" = "6",
    "7" = "7 or more people in the Family"
  ),
  age_cat = cut(
    age,
    breaks = c(-1, 19, 39, 59, 79, 100),
    labels = c(
      "Less than 20 years of age",
      "20 to 39 years of age",
      "40 to 59 years of age",
      "60 to 79 years of age",
      "80 years of age and over")
  )
)
)
#Verify that all the recoded column show the expected categories
head(demo_bp_all_recoded)

```

Step 4: Remove duplicate entries

Let's check to see if there is any duplicates rows and remove them

```
# check if there are duplicates
nrow(unique(demo_bp_all_recoded)) == nrow(demo_bp_all_recoded)

## [1] TRUE
```

Since the number of unique rows is equal to the number of rows in the dataset, we can conclude that **there are no duplicates row in the dataset**

Invalid blood pressure readings

The variable bpx_comment provides explanation about why some of the blood pressure reading are invalid. Let's first, identify cases where blood pressure readings are marked as invalid, and set those associated measurements to missing

```
# identify the codes that indicate invalid blood pressure in the bpx_comment
variable
invalid_bpx <- c(1, 2, 3, 4, 5, 6, 7, 56, 72, 84, 99, 122)
sum(demo_bp_all_recoded$bpx_comment %in% invalid_bpx)
# set bpx measurement to NA
demo_bp_all_recoded <- demo_bp_all_recoded %>%
  mutate(
    across(
      c(bpxsy1, bpxdi1, bpxsy2, bpxdi2, bpxsy3, bpxdi3, bpxsy4, bpxdi4),
      ~ if_else(bpx_comment %in% invalid_bpx, NA, .)
    )
  )
#Verify that the bpx readings with comment that indicates that they aren't va
lid are properly set to NA
demo_bp_all_recoded %>%
  filter(bpx_comment %in% invalid_bpx) %>%
  select(
    seqn, bpx_comment,
    bpxsy1, bpxsy2, bpxsy3, bpxsy4,
    bpxdi1, bpxdi2, bpxdi3, bpxdi4
  )
```

Export

Now let's export our clean dataset

```
export(demo_bp_all_recoded, here("data", "demo_bp_data.rds"))
```

5 - Filtering and reshaping a dataset

```
# filter for adult patients 21 years or older and keep only the column needed
demo_bp_all_adult <- demo_bp_all_recoded %>%
  filter(age >= 21) %>% # the age column has no missing value
  select(age, seqn, sex, bpxsy1, bpxsy2, bpxsy3,
         bpxsy4, bpxdi1, bpxdi2, bpxdi3, bpxdi4)
# Let's look at the resulting dataset format
head(demo_bp_all_adult)
```

##	age	seqn	sex	bpxsy1	bpxsy2	bpxsy3	bpxsy4	bpxdi1	bpxdi2	bpxdi3	bpxdi4
## 1	69	73557	Male	122	114	102	NA	72	76	74	NA
## 2	54	73558	Male	156	160	156	NA	62	80	42	NA
## 3	72	73559	Male	140	140	146	NA	90	76	80	NA
## 4	73	73561	Female	136	134	142	NA	86	88	86	NA
## 5	56	73562	Male	160	158	154	NA	84	82	80	NA
## 6	61	73564	Female	118	124	126	NA	80	80	82	NA

As shown above, the resulting dataset is in wide format because each participant has multiple blood pressure measurements stored in separate columns (bpxsy1, bpxsy2, bpxsy3, bpxsy4, bpxdi1, bpxdi2, bpxdi3, bpxdi4).

We can use `pivot_longer()` to convert the data to long format, where each blood pressure measurement is represented by a separate row.

```
# Convert the dataset from a wide format to a long format
demo_bp_all_adult_long <- demo_bp_all_adult %>%
  pivot_longer(
    cols = c(bpxsy1:bpxsy4, bpxdi1:bpxdi4),
    names_to = "bp_measurement",
    values_to = "bp_reading"
  )
# Lets look at the new format
head(demo_bp_all_adult_long)
```

##	#	A tibble:	6 × 5		
##	age	seqn	sex	bp_measurement	bp_reading
##	<dbl>	<dbl>	<chr>	<chr>	<dbl>
## 1	69	73557	Male	bpxsy1	122
## 2	69	73557	Male	bpxsy2	114
## 3	69	73557	Male	bpxsy3	102
## 4	69	73557	Male	bpxsy4	NA
## 5	69	73557	Male	bpxdi1	72
## 6	69	73557	Male	bpxdi2	76

As shown above, we now have a long format dataset where each patient will have up to 8 rows, one for each `bp_measurement`. The value for each `bp_measurement` is shown in the column `bp_reading`.

Conclusion

In this exercise, I used R to combine and prepare NHANES data from the 2013–2014, 2015–2016, and 2017–2018 waves. I learned how to merge datasets, identify and handle invalid blood pressure measurements, filter the data to include adults aged 21 years and older, and reshape the data from wide to long format. These steps helped me become more familiar with using R for data cleaning and organization and prepared the dataset for further analysis.